Learn-to-harmonise app — Product & Technical Plan

Oct 30, 2026 · @Rohan

Vision & purpose

The app teaches you to harmonise by working on your own melodies. You hum or play a tune, and it helps you find chords for it. It can suggest options with reasons (Create mode) or coach you while you choose them yourself (Learn mode). Both modes run on one harmoniser engine. Chord recognition from full songs is a later, secondary feature.

Origin and selling point: this started because there's no good hands-on way to learn harmonisation. Chord-detection apps tell you what a finished song uses; textbooks teach rules on exercises you don't care about. Nothing takes *your* tune, lets *you* try chords under it, and explains what works and why.

Core promise: "Hum a tune. Find its chords. Learn why they work."

Who it's for (working assumption, to confirm):

- **Primary:** self-taught musicians who can find a melody by ear but don't know how to put chords under it
- **Secondary:** songwriters who have a melody and want fresh chord options, with reasons, to choose from
- **Later:** learners who want to work out the chords of songs they love (needs full-mix chord recognition)

Where the prototype stands today:

Area	State	Notes
Capture	Working	Live mic record + file upload
Note transcription	Decent	Basic Pitch; onsets and pitches read well on clear single lines
Note display	Good foundation	Piano-roll lanes with dots, sustain tails, "show faint" toggle

Area	State	Notes
Key	Partial	Shows a key (e.g. C minor) once enough is played
Chords	Not usable	Built from Basic Pitch's transcribed notes, so it fails on real audio
Harmony theory	Not started	No Roman numerals, function or explanation yet
Character	Present	Purple blob mascot with eyes and smile; idle on home screen, small in results
Feedback curves	Present, unexplained	Three lines (purple, green, orange) under the roll; meaning isn't shown to the user

Two modes, one engine

The app has a creative mode and a teaching mode. Both use the same harmoniser engine and the same melody, so a user can switch mid-take.

	Create mode	Learn mode
User's goal	Get good chords for my tune, fast	Become able to harmonise on my own
Who picks the chords	The app suggests; the user picks among 3	The user picks; the app reacts
What the engine shows	3 full progressions (safe, colourful, surprising), each with reasons	Nothing up front; scores and alternatives after each choice
Feedback style	"Why this works" notes on each chord	"Fits / adds colour / clashes" + hear yours vs. the alternatives
Output	Chord sheet, MIDI, a playable backing	Progress through the learning path; unlocked chord families

Rule for both modes: there's no single right answer. Harmonisation has many valid answers. Feedback is always a spectrum, and it always comes with sound: hearing why it works matters more than being told.

The learning path

Each step unlocks new chords in *both* modes, so Create mode's suggestions grow with what the user has learned. It also gives the gamification real substance.

1. **Find the home note.** Sing the tune and spot where it wants to end. The app reveals the key.
2. **Three chords.** Harmonise with only I, IV and V: pick the one that contains the strong-beat notes.
3. **Endings.** Cadences at phrase ends (V→I to close, ending on V to leave it open).
4. **Substitutes.** vi for I, ii for IV, iii as a softer I; the same melody with a different mood.
5. **Harmonic rhythm.** Choose how often the chords change (one per bar, two per bar, held across two bars).
6. **Colour.** Secondary dominants (V/V, V/vi) and borrowed chords (♭VI, ♭VII, iv in major).
7. **The bass line.** Inversions and passing bass notes for a smooth line.
8. **Voice leading.** Move the inner voices smoothly; avoid parallel fifths only when it matters for the style.

Each step is taught as a short loop: a 20-second demo on a built-in melody, then the same move on the user's own recording, then one challenge.

Experience principles

Every decision should pass these six tests. When two of them conflict, the higher one wins.

1. **It feels like it's listening.** The screen reacts to sound within 50 ms, even before any analysis lands. Silence and sound should look different at a glance.
2. **Show before you tell.** Notes appear as they're played; chords and key settle in as confidence grows. Text explanations come last, and only when asked for.
3. **Confidence is visible.** Uncertain results look uncertain (faint, wobbling, "maybe" chips) instead of being shown as fact. Being honest about doubt builds trust more than guessing confidently.
4. **The character is the emotional channel.** The blob reacts to music (bobs on the beat, leans toward tension, relaxes at a cadence), so the UI chrome can stay quiet.
5. **Every result is playable.** Any chord, note or phrase on screen can be tapped to hear it back. Hearing it is how you understand it.

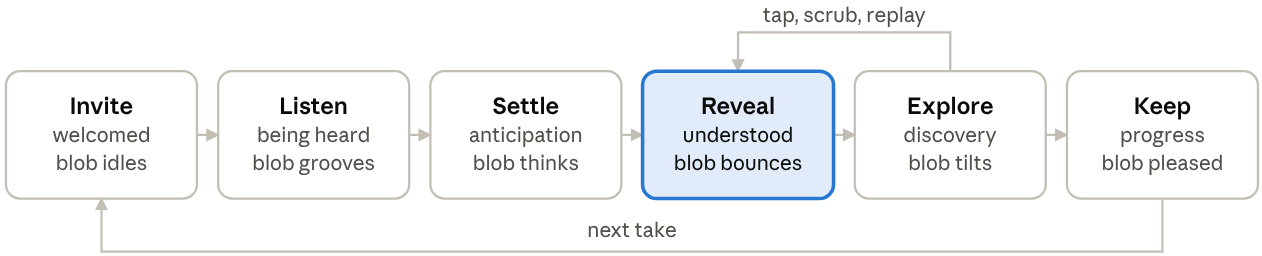
6. **Calm surface, depth on demand.** The default view is minimal (cream canvas, one accent colour). Theory, stems and stats sit one gesture away, never on top.

User journey & experience flow

A session has six beats: **Invite → Listen → Settle → Reveal → Explore → Keep**. Each beat has a job, a feeling, and a character state. The diagram below shows how they connect, and the detail follows it.

Every take moves through six beats, then loops back for the next

Each box: the beat, what the user feels, what the blob does



experience flow · 6 beats, 2 loops

Reveal is the payoff everything builds toward. Explore loops back on it, and Keep feeds the next take.

The fork: after Settle, a melody take forks into **Create** (the app shows progressions) or **Learn** (the user builds one). The default follows the user's last choice. A toggle on the Reveal screen switches between them without re-recording.

1. Invite (home screen)

Job: get the first sound out of the user in under 5 seconds. **Feeling:** a friend waiting to hear you play.

- The blob idles centre-stage: slow breathing scale (0.98 to 1.02 over 4 s), occasional blinks, eyes drift toward the record button.
- The record button is the only strong element. "Play something" sits under the blob. Upload is a quiet secondary action.
- **Auto-listen option:** tapping isn't required. If mic permission is granted, the blob "perks up" (eyes widen, slight hop) when it hears pitched sound, and recording starts. This removes the tap-then-play gap.

- First run only: one line under the blob, "Hum, sing, or play any instrument." No onboarding carousel.

2. Listen (live capture)

Job: prove the app hears every note, in real time. **Feeling:** being heard.

- The canvas fades from cream to a slightly warmer tone. The blob moves up and to the side (as in the current results view) and bobs to the detected beat.
- **Live note trail:** each note lands as a dot on its pitch lane the moment it's detected. Its size reflects loudness and its tail reflects how long it's held. Lanes appear only for pitches actually used, so the roll grows with the music instead of starting as an empty grid.
- **Live "harmonic glow":** a soft colour field behind the roll shifts with the current chord's tension: cool when stable, warmer as tension builds. It's ambient, never a label.
- A small key chip appears only once confidence passes a threshold, as a faint "maybe C minor". It firms up as more evidence arrives. Nothing jumps or flickers. Changes cross-fade over 300 ms, with hysteresis so a chip can't flip back and forth.
- Stopping: tap the button, or 3 s of silence auto-stops after a recording of 10 s or more.

Timing for melody takes: a hummed tune has no drums to find the beat from, and the harmoniser needs to know which notes fall on strong beats. Offer a soft count-in and an optional click (the blob taps its foot), or let the user tap along on a second pass. If neither is used, estimate the beats and mark the grid as "guessed" so the user can nudge the downbeat.

3. Settle (the analysis moment)

Job: cover the 1–5 s that the heavy, non-real-time models need, and make it feel deliberate. **Feeling:** anticipation.

- The live dots "snap" onto the beat grid: they slide to their quantized positions and bar lines draw in from left to right.
- The blob does a "thinking" animation: eyes look up, a slight squash.
- If the full analysis returns something different from the live guess (e.g. the key changes from C minor to E♭ major), the change animates as a morph with a one-line note, never a silent swap.

4. Reveal (the payoff)

Job: deliver the "aha": the key, the progression, and one insight. **Feeling:** being understood.

- **Key headline:** "C minor" with a confidence ring. Tapping it shows the runner-up ("or E♭ major, its relative").
- **Create mode:** three progression cards stacked over the melody roll (*Safe*, *Colourful*, *Surprising*). Each shows chord pills aligned to bars (symbol plus Roman numeral) and a one-line character ("warm and settled", "bittersweet lift", "unexpected turn in bar 3"). Swipe between cards and the backing plays under your melody in a loop.
- **Learn mode:** an empty chord ribbon with a slot per bar. The blob points at the first slot. A small palette shows only the chords unlocked so far. Picking one plays it under the melody straight away and colours the slot (fits / adds colour / clashes).
- **One insight card** (Create) or **one hint** (Learn), never more by default. Examples: "Bar 2 holds a long E: any chord with E in it will feel stable." "Your tune ends on C, so a G before it makes a strong ending."
- For full-song uploads (later), Reveal shows the detected chord ribbon instead, with the same pills and numerals.
- The blob reacts: a satisfied bounce for a clean cadence, a curious head-tilt for something unusual, a wince (playful, never harsh) for a clash.

5. Explore (depth on demand)

Job: let curious users dig in without overwhelming everyone else. **Feeling:** discovery.

- **Tap a chord pill** to hear it alone and under the melody, and open its "why" sheet. This shows the melody notes it contains and the ones it colours, its function (home / away / pull back home), and how it moves from the chord before.
- **Swap a chord:** long-press a pill to see the ranked alternatives for that bar, each with a score bar and a reason. Picking one replays the phrase. This is where Create quietly turns into Learn.
- **Compare:** in Learn mode, "show me yours vs. the app's" plays both versions back to back with the differences highlighted.
- **Tap a note** to see whether it's a chord tone or a passing/neighbour tone under the current chord.
- **Style switch:** pop, folk, jazz, hymn, cinematic. This re-ranks suggestions using that style's progression habits, and the reasons cite it ("common in folk").

- **Lens switcher** over the same timeline: Notes · Chords · Function (tension curve) · Bass line.

6. Keep (save, compare, return)

Job: turn a one-off into a habit. **Feeling:** progress.

- Auto-save every take into a **sketchbook** (cards show the key, the progression and a mini roll as a thumbnail).
- Export: chord sheet (text / PDF), MIDI, and a short shareable clip (audio plus animated roll).
- **Revisit:** re-open a take, re-run analysis with newer models, or record over it to compare ("you played it 8 BPM faster today").

Feature set

Features are split into three tiers. **Core** is what makes the promise true; **Next** deepens it; **Later** is where engagement and gamification live.

Tier	Feature	What it does	Depends on
Core	Melody capture + note trail	Real-time notes on lanes while humming/playing	Monophonic pitch tracking
Core	Count-in, click, tap-along	Gives the melody a reliable beat grid	Simple; Beat This! as fallback
Core	Key finder	Key + runner-up from the melody, shown as "home note" first	Key profile over the melody's notes and durations
Core	Harmoniser engine	Scores every candidate chord per bar; builds whole progressions	Theory rules + progression prior
Core	Create mode	3 progressions (safe / colourful / surprising) with reasons	Harmoniser
Core	Learn mode	User picks per bar; graded feedback; compare with the app	Harmoniser

Tier	Feature	What it does	Depends on
Core	Learning path (steps 1–4)	I–IV–V, cadences, substitutes	Harmoniser + content
Core	Tap to hear / backing playback	Chords played under the user's own recording	Synth / sampler, aligned to the grid
Next	Swap-a-chord	Ranked alternatives per bar with reasons	Harmoniser
Next	Style presets	Pop, folk, jazz, hymn, cinematic priors	Style-tagged progression corpus
Next	Learning path (steps 5–8)	Harmonic rhythm, colour chords, bass, voice leading	Harmoniser + voicing generator
Next	Sketchbook + exports	Saved takes; chord sheet, MIDI, backing audio	Storage
Next	Instrument views	Guitar fretboard / piano diagrams for each chord	Voicing generator
Later	Chords from full songs	Upload a song, see its progression (the original recognition pipeline)	Separation + chord model + decoder
Later	Polyphonic input	Harmonise from a played chord part, or check one	Polyphonic transcription
Later	Progression collection	Unlock named progressions as you use them	Pattern library
Later	Ear challenges	The app plays a melody + chords; you name or play them	Same engine, reversed
Later	Play-along	Loop your progression with drums/bass; jam over it	Generator + live pitch
Later	Blob growth	Cosmetic changes from learning-path progress	—

Gamification guardrail: rewards attach to learning-path progress and *musical discovery* (a new progression, a new key, a first modulation), never to raw time spent. Streaks are gentle: the blob looks pleased to see you, never sad that you left.

Visual & motion language

Keep what's working (cream canvas, one lavender accent, soft pill buttons, lots of air) and give every visual element a musical meaning.

Canvas & layout

- Vertical stack, top to bottom: the blob and key headline, then the chord ribbon, the note roll, the tension curve, and finally controls. It's one timeline shared by every layer, so a vertical line through any moment shows note, chord and function together.
- The roll currently leaves large empty gaps between used lanes (e.g. F2 to A2 in the second screenshot). **Collapse unused lanes** into thin separators, or show a compact keyboard edge so octave jumps still read.
- The three curves under the roll have no labels today. Either label them on hover/tap (e.g. loudness, pitch confidence, harmonic tension) or replace them with **one** tension curve that the user can learn.

Colour as meaning (one hue family per job)

Element	Encoding
Notes	Lavender; opacity = model confidence; size = velocity
Chord function	Tonic = calm blue-violet, predominant = soft green, dominant = warm amber; borrowed/chromatic = rose outline
Key area	Very faint background wash; shifts hue on modulation
Uncertainty	Dashed outline + 50% opacity; never a different hue

Character states

State	Trigger	Animation
Idle	Home, silence	Breathing, blinks, eyes follow cursor/tilt
Perk up	First pitched sound	Eyes widen, small hop
Groove	Beat detected	Bob on beats, squash on downbeats

State	Trigger	Animation
Lean	Tension rising (dominant)	Tilts forward, eyes narrow slightly
Relax	Cadence resolves	Exhale, settle, smile widens
Think	Settle phase	Eyes up, slow squash
Curious	Unusual chord / modulation	Head tilt, one eye bigger
Delighted	New progression unlocked	Bounce + small sparkle burst

Motion rules

- Real-time reactions in 50 ms or less; layout changes 250–350 ms with ease-out; nothing bounces except the blob.
- Motion is tied to musical time when audio is playing (animations land on beats).
- Respect reduced-motion: the blob holds expressions instead of animating, and transitions become cross-fades.

Fix from the current screens: the smile arc overlaps "Play something" on the home screen. Give the label 24 px clearance below the arc.

The harmoniser engine (core)

The harmoniser has three parts. A transparent theory scorer rates how well each chord fits each bar. A progression model trained on a large chord corpus rates how natural each move is. A search over both finds the best whole progressions. No black-box model picks the chords. That keeps every suggestion explainable, and makes the reasons true rather than invented afterwards.

Inputs: melody notes (pitch, onset, duration), the beat grid with beat strength, the key (plus runner-up), the chords unlocked on the learning path, and the chosen style.

How it works

1. **Slot the melody.** Split it into harmony slots (one per bar by default, half-bar optional), and mark phrase ends (long notes, rests, bars 4 and 8).
2. **Generate candidates.** For each slot, take the chord vocabulary allowed by the key and the unlocked set: diatonic triads and sevenths, then secondary dominants and borrowed chords as they unlock.

3. **Score the fit (per slot).** Each melody note is weighted by duration × beat strength, then classed against the chord:
   - chord tone: full credit
   - available tension (9th, 13th, 11th on minor): partial credit, labelled "colour"
   - passing or neighbour note on a weak beat that moves by step: neutral, no penalty
   - clash on a strong or long note (e.g. a 4th over a major chord, a ♭9): penalty
4. **Score the key fit.** Diatonic chords have no cost. Secondary dominants, borrowed chords and chromatic chords cost progressively more, and the style setting scales that cost.
5. **Score the flow (between slots).** Use the progression prior P(next | previous, key, style), learnt from a chord corpus converted to Roman numerals. Add function rules (home → away → pull → home), root-motion bonuses (down a fifth, by step), a cadence bonus at phrase ends, and a penalty for staying static too long.
6. **Search.** A beam search over the slots finds the N best whole progressions. Three diverse ones are then picked out (they must differ in at least a third of the slots):
   - **Safe:** best total score with a high cost on non-diatonic chords
   - **Colourful:** must include at least one colour chord (seventh, borrowed, secondary dominant)
   - **Surprising:** must include one chord the corpus rarely uses here but the melody fits well
7. **Write the reasons.** For each chord, the two biggest score terms become sentences from templates. An LLM may rephrase them in the blob's voice, but it never chooses chords or invents reasons.

```
S(c_1,\dots,c_n) = \sum_{i} \big[\, w_m\,M(c_i, \text{notes}_i) + w_k\,K(c_i)
\,\big] + \sum_{i} \big[\, w_p \log P(c_i \mid c_{i-1}, \text{key},
\text{style}) + w_r\,R(c_{i-1}, c_i) \,\big] + w_c\,C(c_n)
```

In plain words: M is melody fit, K key fit, P the corpus prior, R root motion and C the cadence bonus. The weights (w) start hand-tuned. Later they're learnt from which suggestions users keep.

Worked example: one bar in C major

Melody: **E** (beat 1, half note), **D** (beat 3, eighth), **C** (beat 3½ to 4, held).

Chord	How the melody sits on it	Fit	Reason the app would show
C	E and C are chord tones; D passes between them	Fits	"Your E and C are both in C, so it feels settled. The D just passes through."
Am	E and C are chord tones	Fits	"Same notes, sadder mood: Am shares your E and C."
Fmaj7	C is a chord tone; E is the 7th	Adds colour	"Your long E becomes a dreamy 7th over F."
G	E is a 13th on a strong beat; C is a held 4th	Clashes	"Your held C rubs against G's B. Try it before C instead."
Em	E is a chord tone; the held C rubs against B	Partly clashes	"Starts well, but the C at the end pulls against it."

Learn-mode grading

The user's chord is scored with the same terms and compared with the best options for that slot:

- **Fits:** melody fit near the best, no clash on strong beats
- **Adds colour:** melody fit fine, but it's outside the key or a tension is prominent; the app explains the colour
- **Clashes:** a long or strong-beat note clashes; the app plays the clash and offers the two closest fixes
- Flow feedback is given separately ("Nice ending: G → C closes the phrase"), so a good chord in an odd place is still credited.

Data needed: a chord-progression corpus in Roman numerals (e.g. Chordonomicon, ~666k songs with genre tags, for style priors), a curated rules file (~200 lines), and around 30 built-in melodies for the learning path. None of it needs a trained neural model to ship v1.

Why chord detection is failing today

Priority note: this and the next section matter for the later "chords from full songs" feature and for polyphonic input. The core melody-harmonising flow doesn't depend on them.

The current approach (transcribe notes with Basic Pitch, then look up which chord those notes spell) breaks for five reasons. Each one needs a different fix, so it's worth naming them separately.

1. **Transcription errors compound.** Basic Pitch is tuned to find note *events* (onsets). Sustained or quiet accompaniment, like a pad, a strummed chord ringing out or a held left hand, often produces few or no events, so the chord's notes simply aren't in the list. Meanwhile, overtones show up as ghost notes an octave or a 12th up. Any error in the note list becomes a wrong chord.
2. **A chord isn't a set of simultaneous notes.** A chord is the harmony *implied over a stretch of time*, usually a beat to a bar. An arpeggiated guitar part never sounds two notes together, but it's still one chord. Looking at note overlaps misses this.
3. **Melody and passing notes get counted as chord tones.** If the melody plays a D over a C-minor chord, a note-set lookup says "Cm(add9)" or something worse. Real analysis decides which notes are chord tones and which are passing, neighbour or suspension tones. That decision needs metre (strong vs weak beats) and context.
4. **No timing scaffold.** Without beats and bars, there's no natural place to put chord boundaries, so chords flicker or change mid-beat.
5. **No musical prior.** Some progressions are far more likely than others (V→I vs V→♭II), and the key constrains what's plausible. A note lookup has no memory of the previous chord and no idea of the key, so it can't break ties.

The fix in one line: detect chords *from the audio* with a model trained for chord recognition, segment on the beat grid, and decode the sequence with key and progression priors. Use the transcribed notes only as a secondary signal (melody, voicing and bass), never as the source of truth.

Proposed analysis pipeline & model choices

Run two pipelines. A **fast lane** in the browser gives the live feel. A **deep lane** on a server produces the result we actually show at Reveal. The deep lane is a set of specialist models fused by one beat-synchronous decoder. The diagram under the fusion steps shows the flow.

Step 0: decide what kind of input it is

This is the most important branch, and the app doesn't have it yet. Melody-only is now the primary route, and it goes straight to the harmoniser engine.

- **Monophonic** (humming, singing, a single-line instrument): there are no background chords to detect. The job becomes *implied harmony*, meaning suggest the chords that fit the melody. That's a harmonisation problem (see the harmony layer section), not recognition.
- **Polyphonic solo** (piano, strummed guitar): chord recognition from audio works, and the transcribed notes are fairly reliable too.
- **Full mix** (a song file, a band): separate first, then recognise.

A cheap classifier (polyphony estimate from Basic Pitch's note overlap plus spectral flatness) routes each take. The UI should also *say* which mode it's in ("Melody only — here's harmony that would fit").

Model shortlist

Job	Recommended start	Upgrade / alternative	Runs where	Licence note
Note transcription	Keep Basic Pitch	YourMT3-class multi-instrument models for full mixes	Browser (live) + server	Apache-2.0 (Basic Pitch)
Source separation	HTDemucs (4 stems; a 6-stem variant adds guitar and piano, piano weak)	BS-RoFormer: clearly better bass stem, slower	Server; ONNX build exists for in-browser use	HTDemucs MIT; some RoFormer weights are non-commercial
Beats & downbeats	Beat This! (ISMIR 2024)	madmom DBN for steady 4/4	Server; small ~10 MB ONNX model for browser	MIT
Chord recognition	BTC: outputs timed chord segments; 170-chord or maj/min vocabularies	ChordFormer (conformer, better on rare chords); lv-chordia (large vocabulary incl. jazz chords)	Server	BTC MIT; check others before shipping

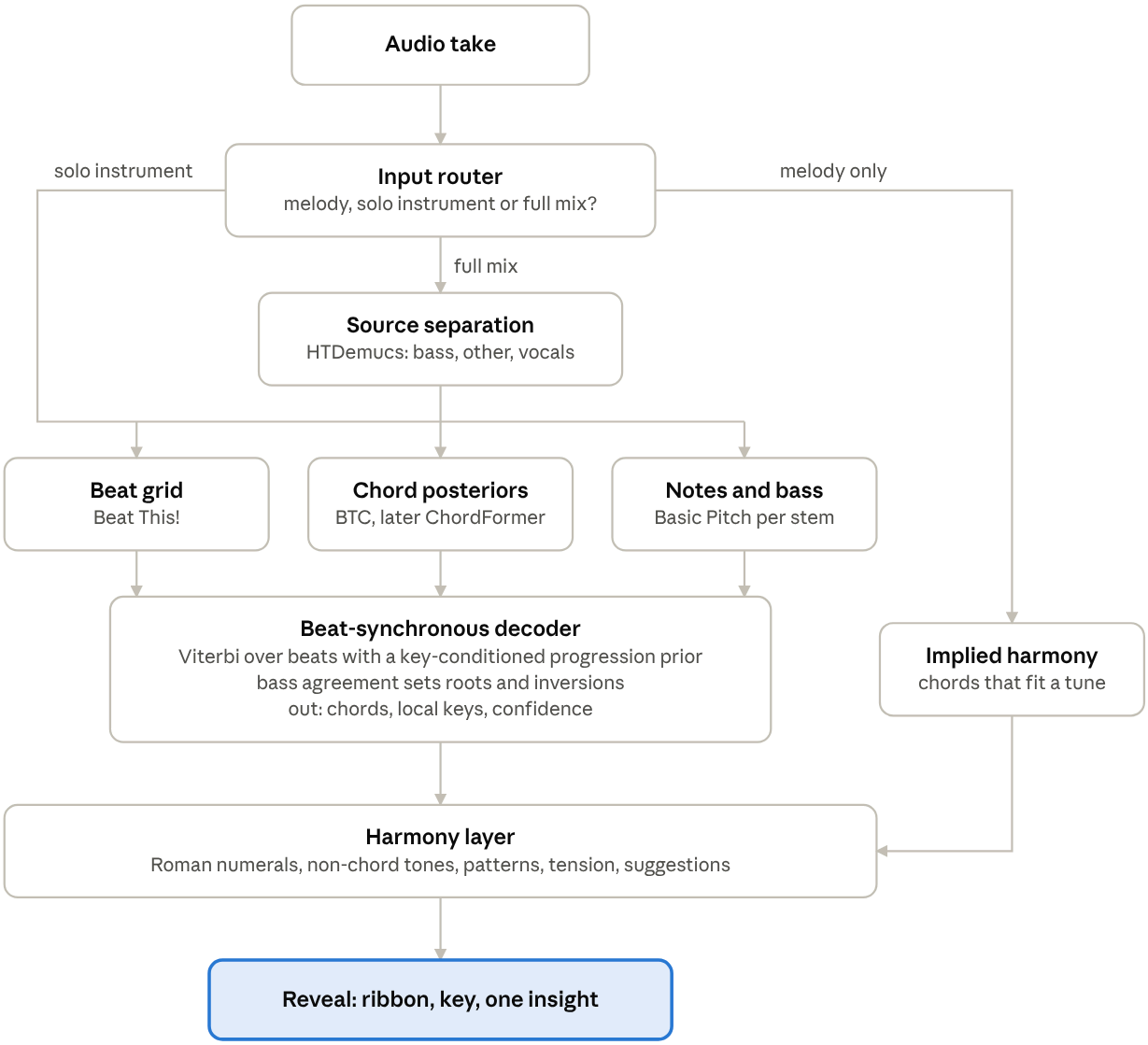
Job	Recommended start	Upgrade / alternative	Runs where	Licence note
Key	madmom CNN key (global)	Derive local key from the chord sequence (HMM over 24 keys)	Server	Check madmom model licence for commercial use
Live chord / key guess	NNLS chroma + chord templates + Viterbi smoothing, Krumhansl key (pitch-detection on npm)	Essentia.js (WASM)	Browser	Essentia is AGPL; check
Roman numerals (symbolic)	Rule-based from chords + key (music21)	AugmentedNet on clean piano transcriptions	Server	—

How the deep lane fuses them

1. **Separate** (full mixes only): HTDemucs gives *bass*, *other* (harmony instruments), *vocals* and *drums*.
2. **Beat grid:** Beat This! on the full mix gives beats, downbeats and tempo.
3. **Chord posteriors:** BTC on the mix (or on *other* + *bass*) gives a probability per chord class per frame.
4. **Bass line:** Basic Pitch on the bass stem (or the lowest transcribed note on a solo instrument) gives the bass note per beat. This sets the inversion (C/E) and often settles the root.
5. **Melody:** Basic Pitch on vocals / the top line. Used to label notes as chord tones or non-chord tones, and for the insight cards.
6. **Beat-synchronous decoding:** average the chord posteriors over each beat, then run a Viterbi/HMM. The emissions are chord posteriors plus agreement with the bass note. The transitions come from a **key-conditioned progression prior**, a bigram/trigram model trained on a large chord-progression corpus (e.g. Chordonomicon, ~666k songs). Chord changes are allowed only on beats and favoured on downbeats.
7. **Key tracking:** a second HMM over the 24 keys, fed by the decoded chords and chroma, gives the global key plus modulations.

8. **Confidence:** keep the posterior margin per segment. Anything below the threshold is shown as "maybe" in the UI.

Chords come from audio models fused on the beat grid, not from the note list



deep-lane analysis pipeline · 3 input routes

Separation runs only for full mixes. Melody-only takes skip recognition and go to implied harmony, and everything meets in the harmony layer.

Why this works: each model does the one thing it's good at. The decoder adds what humans use (metre, key, likely progressions) and nothing is guessed from a single noisy note list.

Fast lane (browser, live)

- Basic Pitch (TF.js) or a lighter monophonic tracker for the note trail.
- Chroma → template chord guess + key, updated every ~250 ms with heavy smoothing, shown as "live guesses".
- An onset-based tempo estimate makes the blob bob. Beat This! small (ONNX) can run on each 5–10 s chunk to refine it.
- When the deep lane returns, its results replace the live guesses with a morph (see Settle).

Harmony theory layer

The harmony layer turns "Cm → A♭ → B♭" into understanding. Most of it is deterministic music theory, not machine learning, which means it's testable and explainable.

1. Roman numerals and function

- Input: decoded chord sequence + local key per segment.
- Output per chord: numeral (i, ♭VI, V^7/V), inversion figure (^{6}, $^6_4$), function (tonic / predominant / dominant), and whether it's diatonic, borrowed (modal mixture), secondary dominant, or chromatic.
- Implementation: a rule engine on top of music21's Roman-numeral tools, with a small set of disambiguation rules (e.g. in minor, prefer VII over ♭VII naming; a major V in minor is the harmonic-minor dominant).

2. Non-chord-tone labelling

- For each melody note, decide: chord tone, passing, neighbour, suspension, appoggiatura or anticipation. The decision uses the beat strength from the beat grid and the step/leap shape.
- This powers insight cards like "that D is a suspension; it resolves down to C on the next beat".

3. Pattern library (the insight engine)

- A curated catalogue of ~50 named patterns, each with a matcher, a one-line explanation and a playable demo. Examples: cadences (authentic, plagal, half, deceptive); common loops (I–V–vi–IV, i–♭VII–♭VI–V Andalusian, ii–V–I, 12-bar blues); devices (pedal point, line cliché, modal interchange, tritone substitution).

- The engine ranks matches by salience (rarer and longer patterns first) and shows the top one as the insight card. The rest are available in Explore.

4. Tension curve

- One number per beat from function (T=low, PD=mid, D=high), dissonance of the chord, non-chord-tone density and distance from tonic key. Drives the colour glow, the blob's lean/relax states, and the Function lens.

5. Implied harmony for melodies (monophonic input)

- This is now the core of the app. See **The harmoniser engine (core)** above.

6. "Try next" suggestions

- Sample the next 1–4 chords from the key-conditioned progression model, then filter with the rule engine for variety: one resolving, one sustaining, one surprising option.

Architecture

The browser owns the live feel. A Python service owns the truth.

Good news for the core flow: the harmoniser is rules + a lookup table (the progression prior is a few MB) + a small search, so it can run fully in the browser. Melody takes need no server at all, so Reveal can be near-instant and work offline. The server is only needed for song uploads (Phase 5) and for syncing the sketchbook.

Layer	Tech (proposed)	Responsibility
Client	Web app (current localhost build), AudioWorklet capture, Web Workers, onnxruntime-web / TF.js	Live note trail, chroma guesses, animation, playback synth
API	FastAPI (Python) + job queue	Accept uploads/recordings, return analysis JSON, stream partial results
Analysis workers	PyTorch: HTDemucs, Beat This!, BTC, Basic Pitch, madmom; GPU optional	Deep lane pipeline
Harmony service	Pure Python (music21 + our rule engine + progression model)	Roman numerals, patterns, tension, suggestions

Layer	Tech (proposed)	Responsibility
Storage	Object store for audio; Postgres for takes and analysis JSON	Sketchbook, re-analysis with newer models

Latency budgets (targets to validate)

Moment	Budget
Sound → dot on screen	⩽ 50 ms visual reaction, ⩽ 150 ms note placement
Live chord/key guess refresh	~250 ms
Stop → Reveal for a 30 s take, no separation	⩽ 3 s
Stop → Reveal with separation (song upload)	Progressive: beats + chords first (⩽ 5 s), stems later

One analysis schema shared by both lanes, so the UI never cares where a result came from: `notes[]`, `beats[]`, `chords[] {start, end, label, roman, function, confidence}`, `keys[]`, `tension[]`, `insights[]`, `source: live|deep`, `modelVersions`.

Privacy: recordings stay on-device unless the user analyses them deeply or saves them. Say so in one line at first save.

Roadmap & evaluation

The roadmap builds the melody path and the harmoniser first, because they carry the selling point and are the more reliable technical problem. Full-song recognition moves to Phase 5. Each phase ends at a measurable gate.

1. **Phase 1: Melody foundation.** Reliable melody capture, count-in/click/tap-along, beat grid, key finder. Test set: 30 hummed, sung and single-line takes with hand-marked beats and key. *Gate:* key correct on 85% or more; strong-beat notes correctly placed on 90% or more.
2. **Phase 2: Harmoniser v1.** Theory scorer + corpus prior + beam search, with reasons from templates. Test set: the same 30 melodies, each harmonised by 2–3 musicians. *Gate:* in blind listening, the Safe suggestion is rated "works" on 80% or more of melodies; at least one of the three matches a human choice on 60% or more of bars.

3. **Phase 3: Create + Learn MVP.** Reveal screen with the mode fork, three progression cards, learn-mode slots with graded feedback, backing playback under the recording, learning-path steps 1–4, blob states. *Gate:* in 5 user sessions, people harmonise a new melody of their own with I–IV–V and explain one choice.
4. **Phase 4: Depth.** Swap-a-chord, style presets, learning-path steps 5–8, sketchbook and exports, instrument views.
5. **Phase 5: Songs.** Full-song chord recognition (separation + beats + chord model + decoder), polyphonic input.
6. **Phase 6: Engagement.** Progression collection, ear challenges, play-along.

Metrics to track

Metric	What it tells us
Key accuracy on melodies (exact + relative/fifth credit)	Is the foundation right?
Strong-beat placement accuracy	Can the harmoniser trust the grid?
Blind "works" rating of Safe / Colourful / Surprising	Are suggestions musically good?
Agreement with human harmonisations (per bar)	Are suggestions sensible, not just legal?
Learn-mode: share of bars rated "fits" over a user's first 10 takes	Are people actually learning?
% of Create takes where the user swaps or keeps a suggestion	Are the reasons and options useful?
Stop → Reveal time (p50 / p90)	Does it feel instant?
Chord accuracy on songs (`mir_eval` , Phase 5)	Is song recognition trustworthy?

Open questions for us to decide together

- [x] ~~**Direction:** a learn-to-harmonise tool with a creative suggestion mode, built on one harmoniser engine~~
- [ ] **Default mode:** should first-time users land in Learn or Create? Create hooks faster; Learn is the promise.

- [ ] **Timing input:** count-in + click by default, or free singing with the beat estimated and nudged afterwards?
- [ ] **Starting styles:** which 2–3 styles first (pop, folk, hymn, jazz)? This decides which corpus slices to build priors from.
- [ ] **Platform:** stay web-first (current build), or wrap for mobile? Singing into a phone is the natural gesture.
- [ ] **Commercial intent:** check licences on the progression corpus and any models before shipping.
- [ ] **Character name and personality:** the blob is now a teacher too. Playful coach, or calm mentor?
- [ ] **Notation literacy:** show Roman numerals by default, or reveal them at learning-path step 3?
- [ ] **The three curves under the roll:** what do they represent today, and do we keep them?

Sources

- BTC chord model (HF packaging)
- ChordFormer (arXiv 2502.11840)
- lv-chordia, large-vocabulary chord transcription
- Beat This! beat tracker and Rust/ONNX port
- HTDemucs, HTDemucs ONNX for browser
- HTDemucs vs BS-RoFormer benchmark
- madmom key recognition
- AugmentedNet
- Essentia.js
- pitch-detection (NNLS chroma, chord templates, key)